

8. 텍스트 분석

NLP이나 텍스트 분석이냐?

NLP(Natural Language Processing): 머신이 인간의 언어를 이해하고 해석하는 데 더 중점을 두고 발전해옴

텍스트 분석(=텍스트 마이닝): 비정형 텍스트에서 의미 있는 정보를 추출하는 것에 더 중점을 두고 발전해옴

NLP는 텍스트 분석을 향상하게 하는 기반 기술이라고 볼 수도 있음.

NLP와 텍스트 분석의 발전 근간에는 머신러닝이 존재

텍스트 분석은 머신러닝, 언어 이해, 통계 등을 활용해 모델을 수립하고 정보를 추출해 비즈니스 인텔리전스(Business Intelligence)나 예측 분석 등의 분석 작업을 주로 수행

1. 텍스트 분석 이해

텍스트: 비정형 데이터

피처 벡터화(추출): 텍스트를 word(또는 word의 일부분) 기반의 다수의 피처로 추출하고 이 피처에 단어 빈도수와 같은 숫자 값을 부여 -> 텍스트는 단어의 조합인 벡터값으로 표현되도록 함

텍스트 분석 수행 프로세스

1. 텍스트 사전 준비작업 (텍스트 전처리): 텍스트를 피처로 만들기 전 아래의 작업들 수행
클렌징, 대/소문자 변경, 특수문자 삭제 등의 클렌징 작업

단어(Word) 등의 토큰화 작업

의미 없는 단어(Stop word) 제거 작업

어근 추출(Stemming/ Lemmatization) 등의 텍스트 정규화 작업

2. 피처 벡터화/추출: 사전 준비 작업으로 가공된 텍스트에서 피처 추출하고 벡터 값 할당

3. ML 모델 수립 및 학습/예측/평가: 피처 벡터화된 데이터 세트에 ML 모델 적용하고 학습/예측/평가 수행

-파이썬 기반 텍스트 분석 패키지

사이킷런: 머신러닝 위주의 라이브러리

=> NLP를 위한 다양한 라이브러리(ex. '어근 처리'와 같은 NLP 패키지에 특화된 라이브러리)는 가지고 있지 X

But 텍스트를 일정 수준으로 가공하고 머신러닝 알고리즘에 텍스트 데이터를 피처로 처리하기 위한 편리한 기능을 제공

=> 사이킷런으로도 충분히 텍스트 분석 기능을 수행 가능

But 더 다양한 텍스트 분석이 적용되어야 하는 경우 보통 NLTK/Gensim/SpaCy와 같은 NLP 전용 패키지와 함께 결합해 애플리케이션을 작성하는 경우가 많음

텍스트 사전 준비 작업(텍스트 전처리) - 텍스트 정규화

텍스트 정규화: 텍스트를 머신러닝 알고리즘이나 NLP 애플리케이션에 입력 데이터로 사용하기 위해 다양한 텍스트 데이터의 사전 작업을 수행하는 것

<종류>

클렌징

토큰화

필터링/스톱 워드 제거/철자 수정

Stemming

Lemmatization

클렌징

텍스트에서 분석에 오히려 방해가 되는 불필요한 문자 기호 등을 사전에 제거하는 작업

ex) HTML, XML 태그나 특정 기호 등을 사전에 제거

텍스트 토큰화

문서에서 문장을 분리하는 문장 토큰화 / 문장에서 단어를 토큰으로 분리하는 단어 토큰화 로 나뉨

NLTK가 다양한 API 제공

문장 토큰화

문장의 마침표(.), 개행문자(\n) 등 문장의 마지막을 뜻하는 기호에 따라 분리하는 것이 일반적

정규 표현식에 따른 문장 토큰화도 가능

```
sent_tokenize
```

```
from nltk import sent_tokenize
```

```
import nltk
```

```
nltk.download('punkt')
```

```
text_sample = 'The Matrix is everywhere its all around us, here even in this room. \
```

```
                You can see it out your window or on your television. \
```

```
                You feel it when you go to work, or go to church or pay your
```

```
taxes.'  
sentences = sent_tokenize(text=text_sample)  
print(type(sentences),len(sentences))  
print(sentences)
```

sent_tokenize의 반환값: 각각의 문장으로 구성된 list 객체

단어 토큰화

기본적으로 공백, 콤마(.), 마침표(.), 개행문자 등으로 단어를 분리

정규 표현식을 이용해 다양한 유형으로 토큰화 수행 가능

단어의 순서가 중요하지 않은 경우 문장 토큰화를 사용하지 않고 단어 토큰화만 사용해도 충분함

일반적으로 문장 토큰화는 각 문장이 가지는 시맨틱적인 의미가 중요한 요소로 사용될 때 사용

word_tokenize

```
from nltk import word_tokenize
```

```
sentence = "The Matrix is everywhere its all around us, here even in this room."  
words = word_tokenize(sentence)  
print(type(words), len(words))  
print(words)
```

sent_tokenize + word_tokenize

```
from nltk import word_tokenize, sent_tokenize
```

#여러개의 문장으로 된 입력 데이터를 문장별로 단어 토큰화 만드는 함수 생성

```
def tokenize_text(text):
```

```

# 문장별로 분리 토큰
sentences = sent_tokenize(text)
# 분리된 문장별 단어 토큰화
word_tokens = [word_tokenize(sentence) for sentence in sentences]
return word_tokens

```

```

#여러 문장들에 대해 문장별 단어 토큰화 수행.
word_tokens = tokenize_text(text_sample)
print(type(word_tokens),len(word_tokens))
print(word_tokens)

```

문장을 단어별로 하나씩 토큰화 할 경우 문맥적인 의미는 무시됨 => 해결책: n-gram

n-gram

연속된 n개의 단어를 하나의 토큰화 단위로 분리해 내는 것
n개 단어 크기 윈도우를 만들어 문장의 처음부터 오른쪽으로 움직이면서 토큰화를 수행함 (슬라이딩 윈도우마냥)
ex) "Agent Smith knocks the door"을 2-gram으로 만들면
(Agent, Smith), (Smith, knocks), (knocks, the), (the, door)

스톱 워드 제거

Stop word: 분석에 큰 의미 없는 단어, 문맥적 큰 의미 없는 단어 (ex. is, the a, will ...)
사전에 제거하지 않으면 빈번함으로 인해 중요한 단어로 인지될 수 있기에 제거가 중요.

NLTK의 English의 경우 몇 개의 stopwords가 있을까?

```

print('영어 stop words 갯수:',len(nltk.corpus.stopwords.words('english')))
print(nltk.corpus.stopwords.words('english')[:20])

```

stopwords를 필터링으로 제거해 분석을 위한 의미 있는 단어만 추출

```
import nltk
```

```
stopwords = nltk.corpus.stopwords.words('english')
```

```
all_tokens = []
```

```

# 위 예제의 3개의 문장별로 얻은 word_tokens list 에 대해 stop word 제거 Loop
for sentence in word_tokens:

```

```
    filtered_words=[]
```

```
    # 개별 문장별로 tokenize된 sentence list에 대해 stop word 제거 Loop

```

```

for word in sentence:
    #소문자로 모두 변환합니다.
    word = word.lower()
    # tokenize 된 개별 word가 stop words 들의 단어에 포함되지 않으면
word_tokens에 추가
    if word not in stopwords:
        filtered_words.append(word)
all_tokens.append(filtered_words)

```

```
print(all_tokens)
```

is, this 등의 스톱 워드가 제거됨!

Stemming과 Lemmatization

Stemming과 Lemmatization: 문법적 또는 의미적으로 변화하는 단어의 원형을 찾는 것
ex) work -> worked, works, working

Stemming: 원형 단어로 변환 시 일반적인 방법을 적용하거나 더 단순화된 방법을 적용해 원래 단어에서 일부 철자가 훼손된 어근 단어를 추출하는 경향

Lemmatization: 품사와 같은 문법적인 요소와 더 의미적인 부분을 감안해 정확한 철자로 된 어근 단어를 찾아줌

=> Lemmatization이 Stemming보다 변환에 더 오랜 시간을 필요로 함

Stemmer: LancasterStemmer - 비교적 단순

```
from nltk.stem import LancasterStemmer
```

```
stemmer = LancasterStemmer()
```

```
print(stemmer.stem('working'),stemmer.stem('works'),stemmer.stem('worked'))
```

```
print(stemmer.stem('amusing'),stemmer.stem('amuses'),stemmer.stem('amused'))
```

```
print(stemmer.stem('happier'),stemmer.stem('happiest'))
```

```
print(stemmer.stem('fancier'),stemmer.stem('fanciest'))
```

work는 단순해서 잘 찾았으나, 나머지는 잘 못 찾음

Lemmatization: WordNetLemmatizer - 비교적 정교

```
from nltk.stem import WordNetLemmatizer
```

```
import nltk
```

```
nltk.download('wordnet')
```

```

lemma = WordNetLemmatizer()
print(lemma.lemmatize('amusing','v'),lemma.lemmatize('amuses','v'),lemma.lemmatize('amused','v'))
print(lemma.lemmatize('happier','a'),lemma.lemmatize('happiest','a'))
print(lemma.lemmatize('fancier','a'),lemma.lemmatize('fanciest','a'))

```

3. Bag of Words - BOW

BOW: 문서가 가지는 모든 단어를 문맥이나 순서를 무시하고 일괄적으로 단어에 대해 빈도 값을 부여해 피쳐 값을 추출하는 모델

양념 감튀 만드는 방법과 유사하다고 해서 이름이 이렇다고 함 ㅎㅎ

다음과 같은 2개의 문장이 있을 때 BOW의 단어 수 기반으로 피쳐를 추출

‘My wife likes to watch baseball games and my daughter likes to watch baseball games too’

‘My wife likes to play baseball’

사이킷런의 Count 및 TF-IDF 벡터화 구현: CountVectorizer, TfidfVectorizer

 한 개의 문서(Document)	Term Frequency							
	The	Matrix	is	nothing	but	an	advertising	gimmick
	40	5	50	12	20	45	3	2
 모든 문서들(Corpus)	Document Frequency							
	The	Matrix	is	nothing	but	an	advertising	gimmick
	2000	190	2300	500	1200	3000	52	12

$$TFIDF_i = TF_i * \log \frac{N}{DF_i}$$

TF_i = 개별 문서에서의 단어 i 빈도

DF_i = 단어 i를 가지고 있는 문서 개수

N = 전체 문서 개수

CountVectorizer 클래스

카운트 기반의 벡터화 구현

피쳐 벡터화 뿐만 아니라 소문자 일괄 변환, 토큰화, 스톱 워드 필터링 등의 텍스트 전처리도 함께 수행

fit()과 transform()을 통해 피쳐 벡터화된 객체를 반환

파라미터 명	파라미터 설명
max_df	전체 문서에 걸쳐서 너무 높은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 너무 높은 빈도수를 가지는 단어는 스톱 워드와 비슷한 문법적인 특성으로 반복적인 단어일 가능성이 높기에 이를 제거하기 위해 사용됩니다. max_df = 100과 같이 정수 값을 가지면 전체 문서에 걸쳐 100개 이하로 나타나는 단어만 피처로 추출합니다. Max_df = 0.95와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐 빈도수 0~95%까지의 단어만 피처로 추출하고 나머지 상위 5%는 피처로 추출하지 않습니다.
min_df	전체 문서에 걸쳐서 너무 낮은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 수백~수천 개의 전체 문서에서 특정 단어가 min_df에 설정된 값보다 적은 빈도수를 가진다면 이 단어는 크게 중요하지 않거나 가비지(garbage)성 단어일 확률이 높습니다. min_df = 2와 같이 정수 값을 가지면 전체 문서에 걸쳐서 2번 이하로 나타나는 단어는 피처로 추출하지 않습니다. min_df = 0.02와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐서 하위 2% 이하의 빈도수를 가지는 단어는 피처로 추출하지 않습니다.
max_features	추출하는 피처의 개수를 제한하며 정수로 값을 지정합니다. 가령 max_features = 2000으로 지정할 경우 가장 높은 빈도를 가지는 단어 순으로 정렬해 2000개까지만 피처로 추출합니다.
stop_words	'english'로 지정하면 영어의 스톱 워드로 지정된 단어는 추출에서 제외합니다.
n_gram_range	Bag of Words 모델의 단어 순서를 어느 정도 보강하기 위한 n_gram 범위를 설정합니다. 튜플 형태로 (범위 최솟값, 범위 최댓값)을 지정합니다. 예를 들어 (1, 1)로 지정하면 토큰화된 단어를 1개씩 피처로 추출합니다. (1, 2)로 지정하면 토큰화된 단어를 1개씩(minimum 1), 그리고 순서대로 2개씩(maximum 2) 묶어서 피처로 추출합니다.
analyzer	피처 추출을 수행한 단위를 지정합니다. 당연히 디폴트는 'word'입니다. Word가 아니라 character의 특정 범위를 피처로 만드는 특정한 경우 등을 적용할 때 사용됩니다.
token_pattern	토큰화를 수행하는 정규 표현식 패턴을 지정합니다. 디폴트 값은 '\b\w\w+\b'로, 공백 또는 개행 문자 등으로 구분된 단어 분리자(\b) 사이의 2문자(문자 또는 숫자, 즉 영숫자) 이상의 단어(word)를 토큰으로 분리합니다. analyzer= 'word'로 설정했을 때만 변경 가능하나 디폴트 값을 변경할 경우는 거의 발생하지 않습니다.
tokenizer	토큰화를 별도의 커스텀 함수로 이용시 적용합니다. 일반적으로 CountTokenizer 클래스에서 어근 변환 시 이를 수행하는 별도의 함수를 tokenizer 파라미터에 적용하면 됩니다.

주요 파라미터!